Nicole Esther Lee designs for whoever's *holding the controller*: a player, a level designer, or someone just trying to sort out lunch.

Currently
SRE intern @ DBS Bank in Singapore

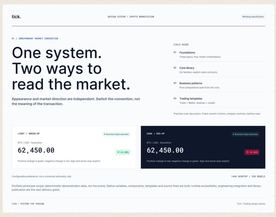
Studying
Computer Science in Interactive Media & Game Development @ SIT–DigiPen

Also
Game designer, level designer and fine arts kid at heart

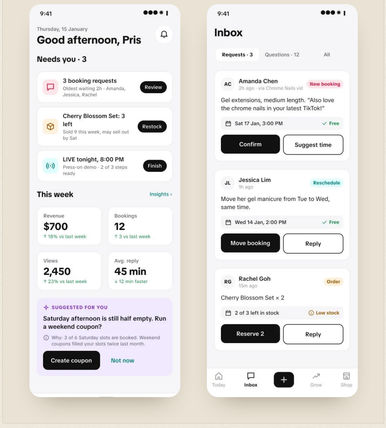
Product & UI/UX

3 projects

Makan Kaki / Mobile app · v3 in progress



Tick / Design system · Case study



Polished by Pris / TikTok design task · v2 rework

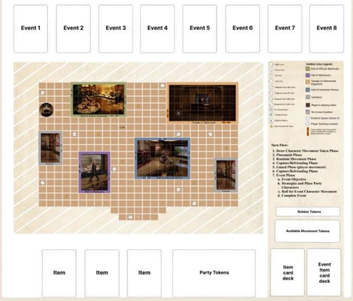
Game design

4 projects

Market Mayhem / Card game · Grade A

Whispers of the Garden / Card game · Grade A

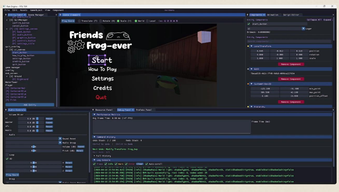
Nyx: Dreams of the Stray / Level design · Grade A



Night at the Museum / Board game · Team project

Game programming

2 projects



Friends Frog-ever / Level editor · Custom C++ engine

Chroma / Physics, tools & team lead

CONTACT

Let's make something *people* use.

nicoleestherlee@gmail.com · Copy email · Résumé (PDF)

say hello

Makan Kaki

Ordering lunch for the team shouldn't mean a 40-message chat and chasing PayNow afterwards.

Role
Product designer (solo)

Duration
UX: 1 month

Stack
Figma, variables & modes

Status
v1.0 progress: Oct 2026

View it here ↗ — FIGJAM · WORKING · LIVE · RESPONSIVE · UI · UX · STYLE · SHEETS · DESIGN LAB

01 — Context

One person does the ordering, then everyone forgets to pay.

In Feb, someone usually volunteers to be lazy for the team. Orders come in through chat, and then add the ingredient three times and have to chase repeatedly.

Goal: Make collecting orders and getting paid back take less effort than doing it in a group chat.

02 — Research

Two people, two very different jobs.

Before drawing anything, I split the problem by who does what. The **organiser** does the most work, but the **colleagues** are the reason it exists. I designed for both, but made sure the organiser's side of it took as little time as possible.

Organiser

"I'll go fuss, everyone send me your orders."

- Collects orders from scattered messages
- Tracks the money
- Works out who owes what
- Chases people to pay

Colleague

"Chicken rice, less lime please and cash."

- Wants to add it in seconds
- Doesn't want another app account
- Forgets to pay back

03 — Core flow

make collecting orders and getting paid back as effortless as group chat, for both the organiser and everyone else!

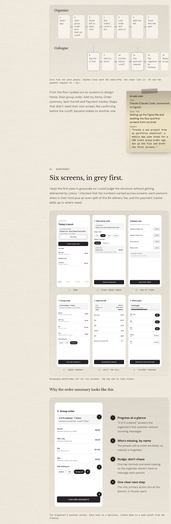
Assumptions I need to test

✅ Validate	Organisers are the ones who would download the app; colleagues will only ever open a link.
⚠️ Risky	Chasing money is a bigger pain than collecting orders.
❌ Invalid	Everyone pays by PayNow, so an in-app wallet isn't needed.
🔍 Unsure	Splitting delivery fees evenly feels fair to people.

04 — Core flow

Mapping the flow before the screens.

I mapped both people's steps side by side. The dashed lines are the key hand-offs, where the app has to save something from one person's flow to show up in the other's. Also the payment request. Those are the moments most likely to break, so they got the most design attention.



Click the flow to open, hover lines and dots for hand-offs. On order card 3 / 8 I set the deadline I missed. Oops.

From the flow I pulled out six screens to design: Home, Start group order, Add my items, Order summary, Split the bill and Payment tracker. Shape of the app was set, now I had to make it fast enough to behave like a LIFF (Line mini app) and not like a website.

Edge cases I designed for

- Double-tap on Submit
- Cancelled an order after it was added
- Forgot to mark someone as paid
- Joined late and wanted to add to an existing order
- Ordered the wrong thing and wanting the fee and that amount back by email

> "Thanks, I ate yesterday. If you're the one who ordered the Milo, I'll pay you for that too. Sorry, I'll be more careful next time."

05 — Wireframes

Six screens, in grey first.

I kept the first pass in greyscale so I could judge the structure without getting distracted by colour. Each screen had one job and one primary button. The organiser's status is their hub plus an own-split of the 6 delivery fee, and the payment reserve adds up to what's owed.

Prototype interactions will be added. Tap, drag, hover to open more info.

Why the order summary looks like this

The organiser's dashboard varies. Split bills in a windows. Context here to a user's point from the drawing.

06 — Style tweaking

From a generic teal app to a kopitiam menu.

The first version looked like any delivery app. It still looked it against mobile and desktop. I wanted something warmer, more local, and more memorable. I pulled in a cream colour, Playfair as the default way to say and Kopitiam's red for the CTA. It felt better, but still a standard app with new paint.

For UX collected feedback a Kopitiam experience, an old-kopitiam menu board, a Southeast Asian "kaleng gaul" drawing and a Lunar New Year set of calendar. That I used the final look and feel. Warm.

Kopitiam

Makan Kaki · kak chat

Nasi Lemak

Chicken Rice

Mee Goreng

Makan where today?

Nasi Lemak

Chicken Rice

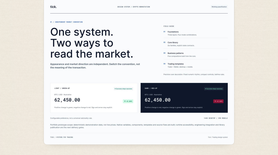
Mee Goreng

DESIGN SYSTEM · CASE STUDY

Tick

A design system for a crypto trading app where colour has to mean the right thing in every market. I wrote the spec, let AI draft it, then audited and fixed it.

ROLE	DATE	TOOLS	TAGS
Design systems	Oct 2026	Figma, Figma AI, variables	Tokens, components, templates



IN THIS PAGE
The problem
The spec
Foundations
Core components
Templates
Explorations
The audit
The fixes
Trade offs

01 — THE PROBLEM

In trading, colour means two things at once.

Most apps use green for success and red for errors. Trading apps also use green and red to price up and down. And in some markets, like mainland China, red means **up**.

A raw red token does both jobs, a normal price drop looks like something broke, and switching to flat-up would turn every error message green. So the whole system rests on one rule:

PRINCIPLE 01

Separate ***value*** *from* ***context.***

A colour value (a hex code) should never decide what something means. Components ask for a meaning, like "price going down" or "error," and the tokens decide the colour for the current mode.

02 — THE SPEC

I wrote the spec before anything was generated.

I defined three token layers, two independent sets of modes, and the components and templates the system needed to support. That spec became the brief for the AI first pass, and the checklist for my audit.

Everything else

PRIMITIVES	SEMANTIC	COMPONENT
Raw colours, spacing, type.	Meaning, per Light or Dark mode.	Only set semantic tokens.
`green/500` · `#1DB954`	`text/success` · `bg/success-subtle`	`button/primary/bg` · `badge/success/text`

Price colours

PRIMITIVES	SEMANTIC	COMPONENT
Raw green and loss colours.	Green-up or Red-up	Price meaning
`green/500` · `#1DB954`	`price/up` · `price/down`	`price/up` · `price/down`

Not a token rename, a statement. Price tokens take an extra hop through the market layer, so components never know which colour means "up."

Two independent switches. Appearance (Light/Dark) and market convention (Green-up/Red-up) are stored separately, so there are four valid combinations, and direction never changes what "success" or "error" feel like.

03 — FOUNDATIONS

Foundations: colour, space, shape and type.

Five primitive colour ramps, a 4px spacing scale, six radius steps and ten type styles. Components never use these directly, they go through the semantic layer.

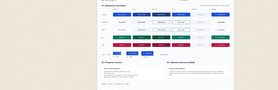
Numbers get their own font. Inter handles interface text, and Roboto Mono handles prices and balances, because equal-width digits keep columns of numbers aligned and stop prices from jumping as they update.

04 — CORE COMPONENTS

Core components, each with a contract.

Six core components: Button, Input, Table, Tag, Toast and Segmented control. Each page documents anatomy, variants, states, tokens and the rules for using it, so designers and engineers make the same decisions.

Button	*Input*	*Tag*
Blue for actions. Green and red for trading, so that trading stays the same size and shape, whether submitting an order.	Units like "BTC" sit outside the field, not inside it, so the value you type and the value you mean "price" never get confused.	"Filled" is always success, "outline" is always error, "ago time status" is price movement.



05 — TEMPLATES

Templates prove the tokens work.

I built a trade and a basic overview in Figma and switched between the two core modes: Light · Green-up and Dark · Red-up. If the tokens were wired correctly, only the colours should change, never the layout or meaning.

Mobile
Mobile
Mobile

Polished by Pris

A TikTok merchant dashboard for a home-based nail technician. My first submission didn't show how I think, so I graded it, then rebuilt it with the research, flows, system and decisions on the pages

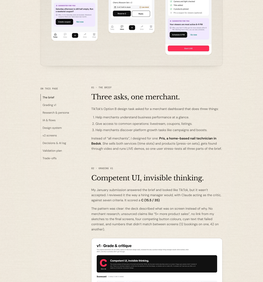
ROLE
Product & UX design (solo)

TOOLS
TikTok merchant dashboard · Figma · Raspberry, Figma Make, Claude

TIMELINE
~1 Jan 2026 - c2 Oct 2026

View in Figma ↗

TIKTOK · UI · ANALYSIS · SYSTEMS · RESEARCH · UX · SYSTEM · BI-DI · DECISIONS · VALIDATION



01 · THE BRIEF

Three asks, one merchant.

TikTok's hybrid strategy team asked for a merchant dashboard that does three things:

1. Help merchants understand business performance at a glance.
2. Give access to common operations: fulfilment, coupons, listings.
3. Help merchants discover platform growth levers like campaigns and lessons.

Rebrand it "all merchants", designed for one **Pris, a home-based nail technician in Singapore**. Her business runs entirely on TikTok Shop: she sells, fulfils, and markets through video and runs LIVE demos, so one user stress-tests all three parts of the brief.

ON THIS PAGE

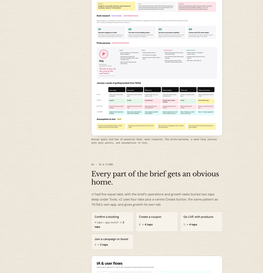
The brief
Grading it
Research & personas
UX & flows
Design system
UI versions
Decisions & N-Ups
Validation plan
Trade-offs

02 · GRADING IT

Competent UI, invisible thinking.

My January submission answered the brief and looked like TikTok, but it wasn't accepted. I reviewed it the way a hiring manager would, with Claude acting as the critic, against seven criteria. It scored **c2 (54.2 / 70)**.

The pattern was clear: the desk described what was on screen instead of why. No research, no personas, no flows, no validation. Every decision was a guess. The fix wasn't to redraw the screens, but completing build-critique-rebuild, each one that failed context, and rundata that didn't match between screens (12 backlogs on rows, 42 on shortlist).



v1 · Grade & critique

Competent UI, invisible thinking.

Overview
Fulfilment
Listings
Coupons
Campaigns
Lessons
Finance
Data
Shop settings

Overall grade
Dimension breakdown
Strengths
Gaps
Rebuild recommendation

Fulfilment
Listings
Coupons
Campaigns
Lessons

Why it scores what it got (and why not enough)

Fulfilment
Listings
Coupons
Campaigns
Lessons

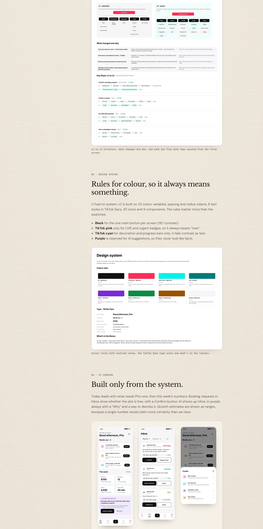
No summary, next arrives fresh: no v2, and no, anyway to remind, slower (those on v2).

02 · RESEARCH & PERSONA

Who Pris is, and what I believed going in.

I used Perplexity for desk research on how home-based nail techs in Singapore find clients and run bookings, and what TikTok already gives sellers. Four patterns kept coming up: discovery happens on video, the inbox is the booking system, services and products are sold together, and promos and LIVEs create urgency.

Honest status: This is a proto-persona built from desk research, not interviews. For assumptions are labelled unvalidated and I speak to real merchants.



Brief, research & persona

Pris
Home-based nail technician · Singapore · TikTok Shop seller

Who she is
Goals
Pains
Behaviours
Why she might be more than just

Pris
Home-based nail technician · Singapore · TikTok Shop seller

Who she is
Goals
Pains
Behaviours
Why she might be more than just

Publications
Publications

Pris
Home-based nail technician · Singapore · TikTok Shop seller

Who she is
Goals
Pains
Behaviours
Why she might be more than just

Publications
Publications

Journey mapped to platform touchpoints (TikTo...

Assumptions to test

Assumptions to test

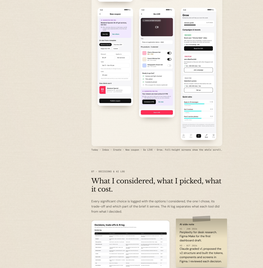
Assumptions to test

design goals and her UI assertive (bias, desk research). No print proposes, 4 week beta (across with open alumni), set assumptions to test.

03 · UX FLOWS

Every part of the brief gets an obvious home.

I'd had five equal tabs, with the brief's operations and growth tasks buried two taps deep under Tools. c2 uses four tabs and a Home Create button, the same pattern as TikTok's own app, and gives growth its own tab.



Confirm a booking
Open booking
Mark complete
Issue refund
Open chat
Edit booking
Tags

Create a coupon
Edit coupon
Pause coupon
Delete coupon

Set up with products
Add a package or bonus!

Create a coupon
Edit coupon
Pause coupon
Delete coupon

UX & user flows

Flow 1 · Fulfil an order

Flow 2 · Create a coupon

Flow 3 · Add a product

Flow 4 · Run an LIVE

Flow 5 · Find a lesson

Flow 6 · Check finance

Flow 7 · Edit shop settings

Flow 8 · Switch to Seller view

Flow 9 · Edit a booking

Flow 10 · Add a booking

Flow 11 · Add a product

Flow 12 · Edit a product

Flow 13 · Add a product

Flow 14 · Edit a product

Flow 15 · Add a product

Flow 16 · Edit a product

Flow

GAME DESIGN · CARD GAME

Market Mayhem

A card game about hype investing, where the stonks are fake but the bad decisions feel real.

RULE	TYPE	TOOLS	RESULT
Game & card design	Tabletop card game	Figma (no AI)	Grade A

ON THIS PAGE

- The idea
- The system
- Achievements
- My expansion
- What I learned
- Trade-offs

01 · THE IDEA

Meme investing, turned into a game.

Market Mayhem borrows from internet investing culture: Stonk Coin, Lambo Cash, Diamond Tokens and MonkeyMania NFTs. Players build a portfolio, gamble on trades and chase achievements before their tokens run out.

I worked on the full game design: rules, card types, the player board and the visual system.

Now it was made
Figma
Figma
Figma

None. Every card, the board and all the artwork were designed by me in Figma.

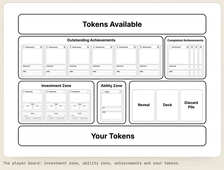
02 · THE SYSTEM

Risk you can read at a glance.

Every security is colour-coded by risk. **Green pays 2×, yellow 3× and red 4×**, so you can judge a card from across the table.

On your turn you either **trade** (go long or short on the next card) or **invest** between a minimum and $10, then roll dice to complete the card's task. Return on Investment cards multiply your stake if you hit the task.

Security cards: green, yellow and red risk levels share one layout.



The player board: investment zone, ability zone, achievements and your tokens.

Return on Investment cards, one per security type and risk level, plus the rainbow "ALL" card and the card back.

03 · ACHIEVEMENTS

Six goals that push you to play differently.

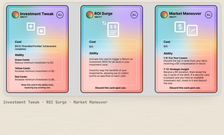
Achievements reward strategies, not luck: hold one of each risk level, diversify across every security or collect three NFTs. Each one unlocks a lasting ability, so finishing one changes how you play the rest of the game.

Achievement cards: Stock Swap, Crypto Trade, NFT Exchange, Dollar Deal, Diversified Portfolio, Risk Manager.

04 · MY EXPANSION

My expansion set.

Each of us also designed a small expansion. Mine adds three ability cards that let players bend the rules: Investment Tweak changes minimum investments by risk colour, ROI Surge cashes in everything at once, and Market Maneuver gives a dice-based gamble.



Investment Tweak · ROI Surge · Market Maneuver

05 · WHAT I LEARNED

Clear rules make chaos fun.

The theme is silly on purpose, but the cards had to be precise. Every number, cost and condition sits in the same place on every card, so players spend their time making bad investments, not reading the rulebook.

06 · TRADE-OFFS

The calls I made, and what they cost.

The theme is chaos. The rules couldn't be.

Decision: How do players read risk?

- ✓ **Colour plus a printed multiplier** — Green, yellow and red read from across the table, and the +2, +3, +4 badge means colour-blind players aren't left out.
- ○ **Numbers only** — Accurate, but players would have to read every card up close.

Decision: How many choices per turn?

- ✓ **Two: trade or invest** — Turns stay fast and easy to teach.
- ○ **Many smaller actions** — More strategy, but slower turns and a much longer rulebook.

Decision: What do achievements give you?

- ✓ **A lasting ability** — Finishing one changes how you play the rest of the game.
- ○ **Just points** — Simpler to explain, but nothing changes once you earn them.

What worked

- Keeping every number in the same spot on every card made the deck quick to scan.
- Abilities from achievements gave players a reason to chase a strategy.

What I'd change

- More cards meant more rules. I'd playtest with new players to see where they get stuck.
- Some ability text is long; I'd tighten it so it fits at a glance.

NEXT PROJECT

Whispers of the Garden →

CONTACT

Let's make something *people* use.

nicoleestherlee@gmail.com · Copy email · Résumé (PDF)

say hello

GAME DESIGN · UNREAL LEVEL

Nyx: Dreams of the Stray

A 3D narrative level about a lonely stray cat, built in Unreal Engine 5 and shaped by six rounds of playtesting.

ROLE	COURSE	TOOLS	RESULT
Solo level designer	DES214 Level Design Methods, DigiPen Redmond	Unreal Engine 5.5, Blueprints, Modeling Tools	Grade A · June 2025

ON THIS PAGE

- The story
- The plan
- The level
- Guidance
- Playtests
- Trade-offs

01 — THE STORY

A stray cat, a white cat, and his past lives.

Nyx is a black stray who falls asleep in an alley and dreams his way through past lives: a cold lab, a busy market, a pampered home. A glowing white cat keeps appearing just ahead of him. At the end, on a rooftop at sunset, he learns who she is and finally rests.

I designed the level as one story arc in six scenes: **setup, hook, three developments, a turn and a resolution.** Every scene had to move the story and the gameplay forward together.

ACT	BEAT	PURPOSE
Setup	Alley	Establish the world and the mood
Hook	Lab	Introduce the white cat
Development 1	Market	Follow the white cat
Development 2	Maze	Lose the white cat
Development 3	Home	Remember the past
Turn	Rooftop	Find the white cat
Resolution	Rooftop	Rest

Six scenes, each built around one spatial archetype. The white cat leads you through all of them.

02 — THE PLAN

I planned every scene on paper first.

For each scene I wrote down the goal, the environment, the obstacles, the spatial archetype and what the player is trying to do, then sketched a rough top-down layout.

My layout sketches: the alley, the dream-state parkour (with the white cat as a guide) and the lab maze.

The market with crowds, the overturned chamber, and the rooftop finale.

A slide from my level plan pitch: the Hook scene, with its goal, obstacles and archetype.

03 — THE LEVEL

Six scenes, one feeling each.

Alley: lonely, quiet, safe

A dim alley at night where you learn to move. Boxes make a winding path to a pillow, and the white cat appears for the first time, then vanishes.

The white cat draws your eye down the path; the pillow is the goal.

Dream state: falling into the dream

A pink haze of floating boxes, planks and bin lids. It's the only real platforming in the level, and the hardest part to balance.

The dream-state parkour, after I made the jumps more forgiving.

Lab maze: trapped

A cold, wet lab you have to escape. Lighting guides you through, but some lit corridors are dead ends on purpose, so you feel lost the way Nyx did. A "Find It" goal sends you into those dead ends to collect memories.

Collectibles were added after playtests showed people wished enough to the exit.

Market, luxury home and rooftop

The market is busy with roaming people. The luxury home is a cat playground with a countdown you can climb anything, but you can't leave until time runs out. The rooftop at sunset is the vista where the story ends.

Left: roaming people make the market feel crowded. Right: the tower in the luxury home.

The rooftop finale. Planks between roofs let you cross to the tower where the white cat waits.

04 — GUIDANCE

Leading the player without telling them.

The white cat. She appears ahead of you, then disappears when you get close, so you always know which way to go next. It also sets up the story twist.

Layout. In the alley, I placed boxes to make a winding path instead of walls.

Light. In the lab, light marks the way out, but some lit areas are dead ends, to make the maze disorienting.

Short text goals. Simple on-screen text, like "Find 3", explains abstract goals in a scene.

05 — PLAYTESTS

Six playtests, three builds.

I ran six playtests with classmates across three builds. I watched where they hesitated, got lost or got frustrated, wrote it down, then fixed the biggest problems before the next build.

PLAYTEST	TIME TO FINISH	FALLS IN PARKOUR
Test 1 Build 1	3:09	5
Test 2 Build 2	3:25	3
Test 3 Build 2	3:56	4
Test 4 Build 3	5:38	2
Test 5 Build 3	7:11	0
Test 6 Build 3	7:48	—

From my playtest reports. Time played more than doubled as the level grew, and falls in the dream-state parkour dropped from 5 to 0 once I simplified the jumps and added respawns. (Test 6 wasn't recorded for both 4.)

What players said

- "The parkour was too difficult, I nearly gave up."
- "I found the maze exit almost immediately."
- "I walked straight through the market."
- "I got stuck and thought the game was bugged."

What I changed

- Simpler jumps and a respawn system in the parkour.
- Collectibles in the maze's dead ends, and a harder-to-find exit.
- Staggered stalls and roaming people in the market.
- Slower movement (500 → 400), and a way back up on the rooftop.

06 — TRADE-OFFS

The calls I made, and what they cost.

Scope was tight, so every scene was a choice between doing more and doing it well.

DECISION
How should players be guided?

- **A white cat that appears and vanishes**
  Guides and tells the story at the same time. Players called it "visually confusing."
- Arrows, signs or a waypoint
  Clearer, but it would break the dreamy mood and say nothing about the story.

DECISION
How hard should the parkour be?

- **Forgiving jumps plus respawns**
  The level is about feeling, not skill. After the changes, falls went from 5 to 0 and testers called it "challenging but fair."
- Keep it hard, like a platformer
  Some players liked the challenge, but one said they would have quit.

DECISION
What happens in the market?

- **Explorable stalls and roaming people**
  Believable in the time I had, and it made the space feel alive.
- The rhythm-mini-game from my plan
  More interactive, but I ran it to finish every scene instead of leaving the last two broken.

What worked

- Playtesting every build: three early testers got stuck in unfinished scenes, which pushed me to finish the whole level before polishing.
- The lab maze. Testers said it "made sense" as a lab or shelter, and the collectibles got them exploring dead ends.
- The rooftop vista. One tester stopped to admire the sunset.

What I'd change

- The market still ends too quickly; it needs a light mechanic or objective.
- Testers finished exploring the luxury home before the timer ran out. I'd shorten the timer or add more to find.
- The rooftop ending felt underwhelming to one tester. I'd build a stronger final moment with the white cat.

NEXT PROJECT

Friends Frog-ever →

CONTACT

Let's make something people use.

nicolesetterlee@gmail.com · Copy email · Résumé (PDF)

say hello

GAME DESIGN · CARD GAME

Whispers of the Garden

A cooperative card game that helps early childhood educators understand and support neurodivergent children, one plant at a time.

ROLE	AUDIENCE	TOOLS	RESULT
Game, card & print design	Early childhood educators in training	Figma, AI image generation	Grade A

FEATURED · NOV 2024

Our module lecturer wrote about the final presentations for Design Innovation (UDE2000) at BIT, and closed with our tagline:

"Nurture Every Bloom, Every Mind."

Read the LinkedIn post ↗

ON THIS PAGE
The idea
Research & testing
The plants
How it plays
The look
Trade-offs
Final presentation

01 · THE IDEA

Nobody wins unless every plant grows.

Players are caretakers in an enchanted garden. Each plant has different needs, and if a single plant withers, everyone loses. To win, the whole table has to work together and pay attention to each plant.

It's built for current and future early childhood educators. Each plant stands in for a child with different needs, so the game becomes a low-stakes way to practise managing a classroom where every child is understood: sharing limited resources, giving individual support and adapting when things go wrong. Cooperation, patience and empathy are the rules of the game, not just the message.

02 · RESEARCH & TESTING

We started with the people in the classroom.

Our team, Walnut Studios, spoke to early childhood educators and trainees. Their pain points were about real moments: not knowing how to adapt when a child isn't responding to an intervention, finding it hard to spot developmental differences in practice, and struggling to stay calm when a child is having a hard time. They also told us what would help: collaborative games, real-life scenarios where choices lead to different outcomes, and more discussion.

> *"A real-life scenario would be good as it would let us know what choices we make would lead to different outcomes."*

From that we built a persona, Emma, a final-year child psychology and early education student on attachment at a childcare centre, and mapped her day before and after the game. It led to our question:

HOW MIGHT WE

provide early childhood educators with a tangible understanding of how psychological interventions can positively make a difference in children's lives?

What testing changed

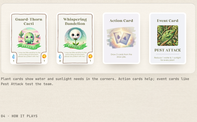
We tested a paper prototype of the character, event and action cards. Players said it felt like nurturing their students, but also that the characters were "quite boring", some cards weren't intuitive, and the goal felt repetitive. So we gave each plant its own dialogue and abilities, made event cards more random, and added more action cards to help the game progress, with clearer descriptions.

03 · THE PLANTS

Five plants, each with their own needs.

Each plant gently represents a different condition, such as ADHD, autism, OCD or social anxiety. The Brightburst Sunflower bursts with energy, and the Zone Out Lily focuses deeply on one thing.

Every plant card asks the same thing in its own voice: **"please be patient with me."** There are 5 plant types with 3 growth levels each, and each one ends up immune to a different hazard.



Plant cards show water and sunlight needs in the corners. Action cards help; event cards like Pest Attack hurt the team.

04 · HOW IT PLAYS

Water, sunlight and a bit of weather.

Players share water cards (hydration and calm) and sunlight cards (guidance and stimulation) to help plants level up. Action cards let players trade, protect plants or double a resource for a round.

At the end of each round an event card hits the garden: a hurricane, a drought, or a sunny "Rise & Shine". Too much or too little of anything can set a plant back, just like real care.

Gameplay walkthrough (2:44). Levelling up plants, using action cards and resolving event cards. Edited by me; voice-over by my groupmate Sherrayne.

RULEBOOK

I designed it to be printed: rulebook, card sheets and a fold-up box.

05 · THE LOOK

Soft, warm and never scary.

For a young audience I went with watercolour illustrations, rounded cards and calm greens. The plants have friendly faces so children connect with them as characters, not as labels. I generated the illustrations with AI and designed every card, layout and printable around them.

AI side note

Tool: DALL·E

Prompt: **All 15 watercolour illustrations: plants, events, actions and card backgrounds.**

```
"A whimsical, fantasy-themed illustration of a baby magical cactus. The cactus has a cute, adorable, silly face glowing thorns…" then "…a teenage magical cactus, based on the baby cactus style."
```

06 · TRADE-OFFS

The calls I made, and what they cost.

Designing about neurodivergent children meant every choice had to be gentle as well as fun.

DECISION

Compete or cooperate?

- ✓ **Fully cooperative** — Everyone wins or loses together, so the game rewards patience and helping, not beating each other.
- ✗ **Competitive** — More tension, but it would work against the message of the game.

DECISION

How should the conditions appear?

- ✓ **Plants with faces and their own needs** — Children connect with characters, and the condition is a need to care for, not a label.
- ✗ **Name the conditions on the cards** — Clearer for adults, but it risks turning a child into a diagnosis.

DECISION

How should the art be made?

- ✓ **AI watercolours, prompted as a series** — 35 consistent illustrations in about a week. I kept the style steady by referring back, e.g. "based on the baby cactus style".
- ✗ **Draw everything by hand** — More personal, but not possible in the project time with this many cards.

What worked

- Cooperative play made the whole table talk about what each plant needs.
- Writing prompts as a series kept the illustrations looking like one set.

What I'd change

- I'd test the plant–condition pairings with special needs educators and parents, not only trainees.
- With more time, I'd paint the key plants myself and use AI only for backgrounds.

07 · FINAL PRESENTATION

The deck we presented.

Our final presentation for the module: focus, research, persona, problem statement, ideation, prototyping, testing, the hi-fi cards and what's next.

Walnut Studios: Nicole Esther Lee, Dawn Tan, Wenbinne Gan, Si Ti Zian Nang, Sherrayne Tay, Nolan, use the arrows, or click a slide to see it larger.

NEXT PROJECT

Night at the Museum →

CONTACT

Let's make something *people* use.

nicoleesthertee@gmail.com · Copy email · Résumé (PDF)

say hello

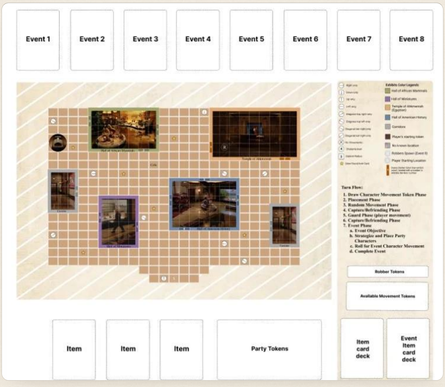
GAME DESIGN · BOARD GAME

Night at the Museum

The brief was a movie-centric board game. Our team chose *Night at the Museum* and turned the museum after dark into a board, a deck of events and a set of items that each bend the rules.

TYPE	TEAM	TOOLS	OUTPUT
Tabletop board game	Group project	Figma (no AI)	Board, cards, 40 tokens, printables

View in Figma ↗ PAGES: CARDS, TOKENS, BOARD, DLC, A4 PRINTABLES, GREEN LIGHT



ON THIS PAGE

- The board
- A turn
- Events
- Items
- The full kit

01 · THE BOARD

The museum, as a grid.

The board is the museum floor laid out as a grid, with the film's exhibits as colour-coded zones: the Hall of African Mammals, the Hall of Miniatures, the Egyptian Temple of Ahkmenrah, the Hall of American History and the corridors between them.

Characters don't move freely. Movement tokens fix the direction (right only, up only, diagonal, and so on), so getting across the museum takes planning. The legend on the side explains every token, the exhibit colours, the player start and where the robbers spawn.

02 · A TURN

Seven phases, printed on the board.

The turn flow sits on the board itself, so nobody has to dig through a rulebook mid-game:

1. Draw character movement tokens
2. Placement
3. Random movement
4. Capture or befriend
5. Guard phase (the player moves)
6. Capture or befriend
7. Event: read the objective, place party characters, roll for the event character's movement, then complete the event

03 · EVENTS

Every event is a front page.

The eight events are styled as issues of *Museum News*, each with a headline from the film: Theodore returns to his exhibit, Columbus sets sail, Ancient Anubis awakens, a massive animal stampede, robbers break into the museum. Each card states the event and its reward, either an item or a new party member.

Museum News vol. 1–8 and the event card back.

04 · ITEMS

Items with abilities and a quote.

Found items, like the Toy Keys, Gum Gum, Guard's Flashlight and Museum Keys, give active or passive abilities, from moving Larry to Dum Dum to looking at an extra card. Each one carries a line from the film. Starting and event items, such as the Tablet of Ahkmenrah and the Spear of Anubis, tie back to the events.

Found items: Toy Keys, Gum Gum, Guard's Flashlight, Miniature Pistol, Museum Keys.

05 · THE FULL KIT

Built to be played, not just shown.

Beyond the board and cards, the file has a sheet of 40 movement tokens, a DLC expansion, A4 printables so the game could be cut out and playtested, and the Green Light pitch we used to get the concept approved.

How it was made

TOOLS
Figma

AI
None. All the designs were done by me in Figma.

See the board, every card, the tokens and the DLC.

Open the Night at the Museum Figma file ↗

NEXT PROJECT

Nyx: Dreams of the Stray →

CONTACT

Let's make something *people* use.

nicoleestherlee@gmail.com · Copy email · Résumé (PDF)

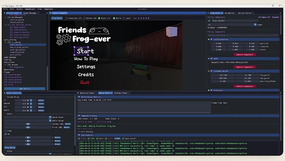
say hello

GAME DEVELOPMENT · TOOLS & UX

Friends Frog-ever

Our team wrote its own C++ game engine. I built the level editor on top of it, modelled on Unity, so our two design students could build a 3D stealth game without writing engine code.

MY ROLE	TEAM	TOOLS	SHIPPED
Built the full level editor	Team F, with 2 design students as my users	C++, custom engine, ImGui, Lua scripting	Windows and Android · Aug 2025 – Apr 2026



ON THIS PAGE

- The problem
- The editor
- Trade-offs
- The game
- What I learned

01 · THE PROBLEM

Two designers, and an engine only programmers could use.

Friends Frog-ever (first called Ribbit Express) is a 3D stealth game: a little frog sneaks through a shipping yard, past patrolling robots, to fix its robot friend. It was a school module, and the brief required a multi-platform game built on our own C++ engine, not Unity or Unreal.

That created a problem. Without an editor, every change to a level, like moving a crate, adjusting a light or wiring up a menu button, meant a programmer editing code and rebuilding. With **two design students** responsible for building the levels, the programmers would become the bottleneck for everything.

MY THINKING

My users weren't players. They were my two teammates. They already knew how to build levels in Unity from class, so the fastest way to make them productive was to give them something that felt like Unity, not a new tool they'd have to learn mid-project.

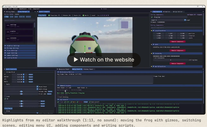
02 · THE EDITOR

Everything a designer needs, in one window.

I built the whole editor: every panel, the tools inside them, and how they talk to the engine. Designers could build a scene, test it and fix it without leaving the window.

Selecting the Frog: move, rotate and scale it in the viewport, and edit its position, rotation and scale exactly in the Inspector.

- **Scene viewport:** play the scene in place, move, rotate and scale objects with gizmos, switch between world and local space, and jump between five camera views.
- **Hierarchy and inspector:** every object in the scene as a tree, with parents and children. Select one to add or remove components and set its layer, just like Unity.
- **Resource panel and prefabs:** browse models, textures, audio and scripts by folder, and reuse objects as prefabs.
- **Script editor:** open and edit the game's Lua scripts right next to the scene, or send them to VS Code.
- **Scene manager:** create, load and save scenes such as the menu, settings, credits and cutscenes.
- **Debug panel:** frame time, an undo and redo history for every edit, and a log console filtered by trace, info, warning and error.
- **Audio controls:** a volume mixer for UI, music and sound effects, with 3D sound settings.



Highlights from my editor walkthrough (1:32, no sound): moving the Frog with gizmos, switching scenes, editing menu UI, making components and writing scripts.

Left: components work like Unity's, added and configured per object. Right: enemy AI scripts edited inside the editor, with every script file one click away.

Level 3 has UIs entities. Designers set physics and colliders in the inspector, and Lua sort the Android touch controls in the same viewport.

03 · TRADE-OFFS

The calls I made, and what they cost.

SESSION

What should the editor look and feel like?

- ✓ **Mirror Unity: same panels, same words**
  Hierarchy, inspector, components, prefabs. The designers already knew where everything was, so they could start building straight away.
  Costs: some Unity ideas didn't map neatly onto our engine, so I had to bend the engine to fit.
- → **Design our own layout around the engine**
  Could fit the engine more naturally, but the designers would be learning a new tool while also building levels.

SESSION

Where should designers edit scripts?

- ✓ **Inside the editor, next to the scene**
  Change an enemy's behaviour and test it without switching windows. Costs a second, simpler code editor to build and maintain, so I added a shortcut to VS Code for bigger edits.
- → **Only in an external code editor**
  Less for me to build, but constant switching between tools slows down the try, test, fix loop that level design depends on.

SESSION

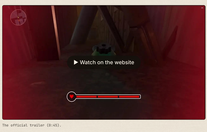
How safe should it be to experiment?

- ✓ **Undo and redo for every edit**
  Designers could try things without fear of breaking the level. Costs: every change had to go through one command system, which made each new tool more work to add.
- → **Save often and hope**
  Simpler to build, but one bad drag could cost a designer their afternoon.

04 · THE GAME

What the designers built with it.

The levels, menus and cutscenes in Friends Frog-ever were put together in the editor. The game shipped on **Windows and Android**, with touch controls on mobile.



The official trailer (0:45).

Fog cuts what you can see. When a robot spots you, the Detected meter fills.

Left: Windows. Right: the Android build with touch controls.

Team F.

05 · WHAT I LEARNED

Building tools is designing for people.

Familiar beats clever. Copying a layout the designers already knew saved more time than any new idea would have.

Undo makes people brave. When mistakes are cheap, designers try more things, and the levels get better.

Sit with your users. The best requests came from watching the designers work, then turning what slowed them down into the next feature.

NEXT PROJECT

Chroma →

CONTACT

Let's make something people use.

nicoleesttherlee@gmail.com · Copy email · Résumé (PDF)

say hello

GAME DEVELOPMENT · PHYSICS, TOOLS & TEAM LEAD

Chroma

A 2D puzzle platformer where you shoot paint to change what platforms do. I joined as physics lead. When parts of the project stalled, I also built the graphics, the level editor and the particle system, and ended up running the team.

MY ROLE	TEAM	TOOLS	SHIPPED
Physics lead, later credited as project manager	Elysium Studios: 6 programmers, a designer and an artist	C++, our own 2D engine	Aug 2024 – Apr 2025

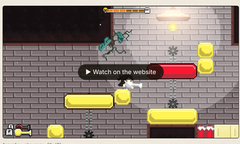
ON THIS PAGE

- The game
- The physics
- Picking up the slack
- Trade-offs
- What I learned

01 · THE GAME

Colour is the puzzle.

Chroma was a school module: the brief required our own 2D engine, written in C++. You play an artist trapped in a dreamscape. Shoot a colour at a slimy paint platform and it changes how it behaves. Collect 6 of the 10 light bulbs in a level to unlock the exit.



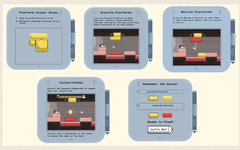
Gameplay showcase (0:47).

02 · THE PHYSICS

Rules simple enough for a tutorial page.

As physics lead, I owned how the platforms move and react. Every puzzle in the game rests on three rules:

- **Same colours stick.** Two yellow platforms that touch join together. Different colours don't.
- **Gravity platforms fall** when you click them, and keep falling until they land on a platform of the same colour.
- **Moving platforms** start and stop when you click them, like a lift.



The in-game tutorial cards. If a rule couldn't fit on one card, it was too complicated.

BY THINKING

Players never see physics code, only whether it feels fair. A platform that stuck to the wrong colour even once made players stop trusting the rules, so getting the edge cases right mattered more than adding new platform types.

Left: shooting paint changes a platform's colour. Right: same-coloured platforms join into one shape.

03 · PICKING UP THE SLACK

From one system to most of the engine.

Partway through, some tasks weren't getting done. Rather than let the game slip, I took them on:

- **Graphics:** how sprites, lighting and backgrounds are drawn on screen.
- **Level editor:** so our designer could lay out levels without asking a programmer.
- **Particle system:** the paint splashes, dust and sparkles that make the game feel alive.

I also started organising everyone's tasks, turning what our designer needed into work for the programmers, fixing bugs and cleaning up tangled code so new features stopped breaking old ones. By the end, my professor credited me as the project manager.

The intro and ending cutscenes, drawn by our artist. The colour coming back is the story.

04 · TRADE-OFFS

The calls I made, and what they cost.

DECISION

Parts of the game weren't being built. What should I do?

- ✓ **Build them myself**
  The graphics, level editor and particle system got done, and the game shipped complete. Costs: far less time to polish my own physics work, and a lot less sleep.
- ✗ **Wait and escalate**
  Fairer on paper, but the deadline wouldn't move, and the whole team's grade depended on a working game.

DECISION

How should the team keep track of work?

- ✓ **One shared task list, with me as the go-between**
  Everyone could see what was left and who owned it, and our designer had one person to ask instead of six. Costs: coordinating took time I could have spent coding.
- ✗ **Let each person manage their own tasks**
  Less overhead, but that's how work had been falling through the cracks.

DECISION

New features kept breaking old ones. Fix or push on?

- ✓ **Clean up the messiest code first**
  Untangling it made each later feature faster and safer to add. Costs: a few days with nothing new to show.
- ✗ **Keep adding features on top**
  More progress early, but every bug fix risked creating two more.

05 · WHAT I LEARNED

A team project is a people project.

Someone has to hold the whole picture. Most of our problems weren't code. They were tasks nobody owned and needs nobody passed on.

Track assets in one place. In our post-mortem, we agreed a shared sheet of every asset, and what's in the game, would have saved hours of "is this done?"

Pull before you push. Most of our merge conflicts came from out-of-date branches. A team habit these that better than any tool.

Elysium Studios, in matching red bows like our main character's.

NEXT PROJECT

Makan Kaki →

CONTACT

Let's make something *people* use.

nicoleesthertee@gmail.com · Copy email · Résumé (PDF)

say hello

Hi, I'm Nicole. I design and build things, and off the clock I *draw, take photos and play.*

Based in
Singapore, camera in hand

Always carrying
A pen, a sketchbook
and my camera

Currently learning
Piano, self-taught,
one wrong note at a time

Jack of all trades, master of ~~most~~ *some*.

CHARACTER SHEET

Okay, maybe a few more than some. Here's where my points are spent so far.

Fine art LV. 8
Where it all started. Art at O and A Levels, and still the reason I care how things look.

Product & UI/UX LV. 8
Personas, flows and hi-fi prototypes in Figma. Lots of school projects, now my main quest.

Game & card design LV. 8
Two card games, two A grades. I promise I'm also fun at game night.

Engine programming LV. 7
C++ engines, physics and level editors, built with two engine teams.

Design systems LV. 7
Tokens, modes and components in Figma, checked line by line.

Automation & SRE LV. 6
Bash, Power Automate and ELK at DBS. Levelling up on the job.

How I *work*

DESIGNER WHO SHIPS

01
Build the tool first
On two game-engine teams, designers needed editors before they could make levels. I learned to design for the person using the tool every day.

02
Audit, don't trust
AI can draft fast. I check the numbers, states and edge cases it gets wrong, and I write down what I fixed and why.

03
Design for here
PayNow, hawker queues, Singlish microcopy. Local details make a product feel like it belongs.

Experience & education

SELECTED

WORK

MAY 2024 – NOW
Site Reliability Engineer Intern · DBS Bank
Global Financial Markets Technology. Automating report runs, job schedules and email alerts with Bash and Power Automate; monitoring with ELK; UAT testing.

EDUCATION

2023 – 2027
Singapore Institute of Technology – DigiPen Institute of Technology
Bachelor of Science in Computer Science in Interactive Media and Game Development.

2020 – 2022
Millennia Institute
GCE A-Levels, including Art. Before that, Art at O Level too.

Off the *clock*

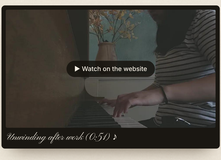
SIDE QUESTS

When I'm not pushing pixels, you'll find me with a pen in hand, out with my camera snapping whatever catches my eye, at the piano working out a song by ear, or wandering around a museum.

SIDE QUEST 01
Pens & colour pencils
I draw with dry media, mostly ballpoint, fineliners and colour pencils. Slow, scratchy and very forgiving of a shaky hand. Flowers keep sneaking in, including the same orchids I photograph.

SIDE QUEST 02
Things that move
For my A Level art coursework I experimented with animation: tracing a figure frame by frame until a few white lines start to run, kneel and wave. It was my first taste of designing for motion.

Line animation experiment, A Level Art (O/O)

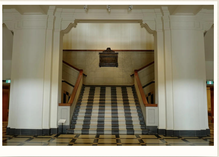


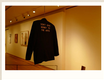
Unwinding after work (O/O)

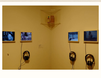
SIDE QUEST 03
Piano, self-taught
To unwind, I sit down at the piano. No lessons, just patience, YouTube and a lot of replaying the same bar. Please be kind!

SIDE QUEST 04
Camera roll
I take my camera everywhere and take pics of whatever makes me stop: orchids, staircases, the odd suspicious bottle in a gallery corner, and the moments that matter to the people I love. You be the judge haha.

Camera roll — 9 photos







Fun fact
I've visited the National Gallery Singapore over 20 times. Every time I go back, I notice something new, or see an old favourite from a new angle.
maybe Pinterest can't give me

CONTACT

Let's make something *people* use.

nicoleesthertee@gmail.com · Copy email · Résumé (PDF)

say hello